

Ph. D. Juan David Martínez Vargas
Ph. D. Raul Andrés Castañeda

Escuela de Ciencias Aplicadas e Ingeniería

Lecture 13

Introducción a

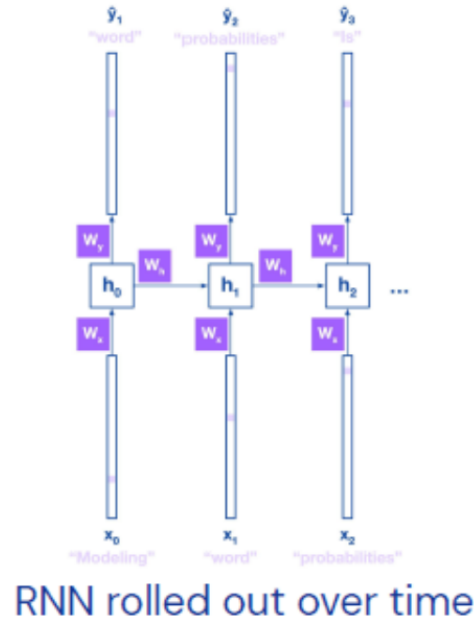
Transformers

Redes recurrentes

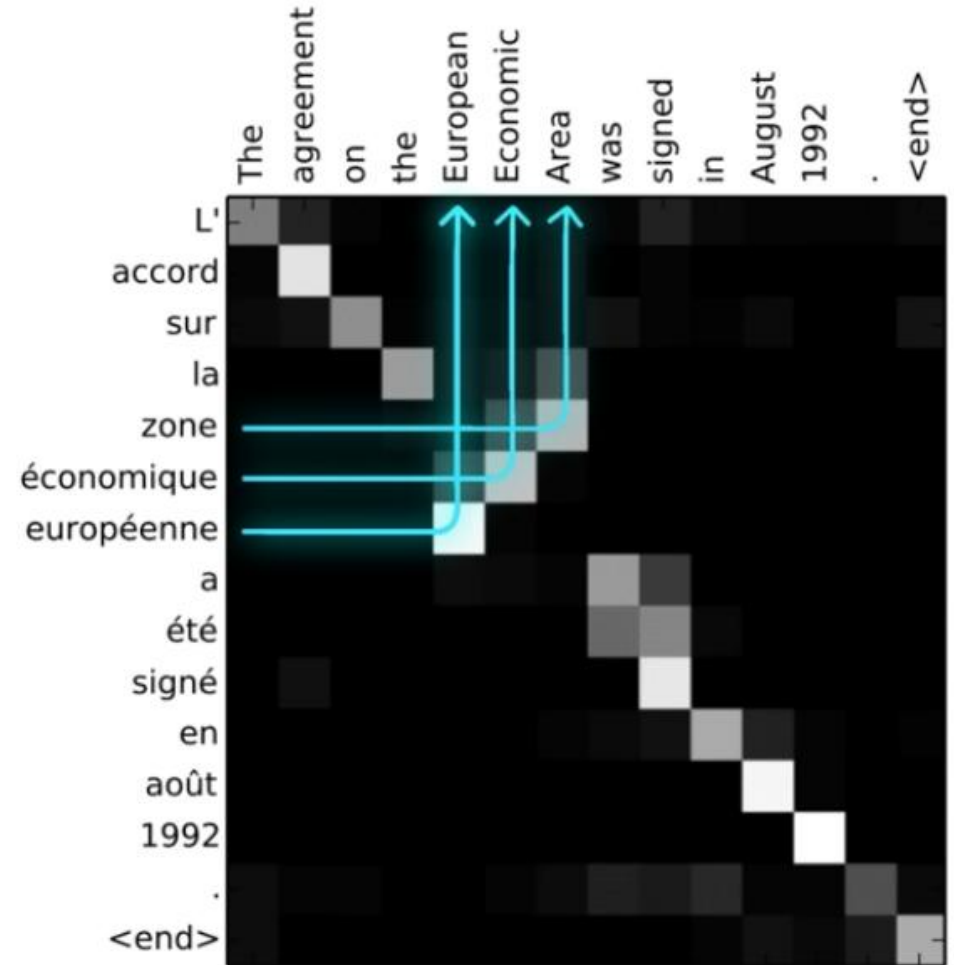
Weights are shared over time steps



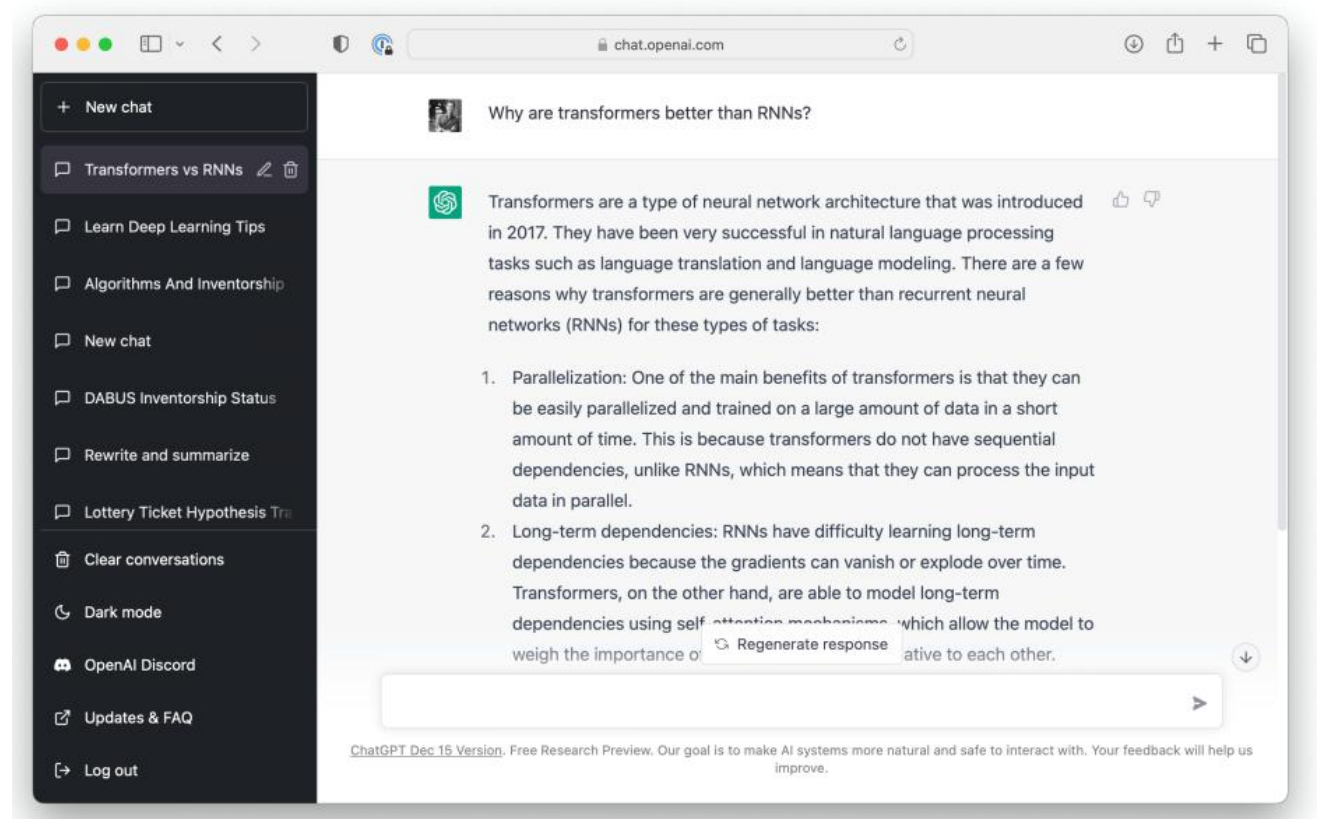
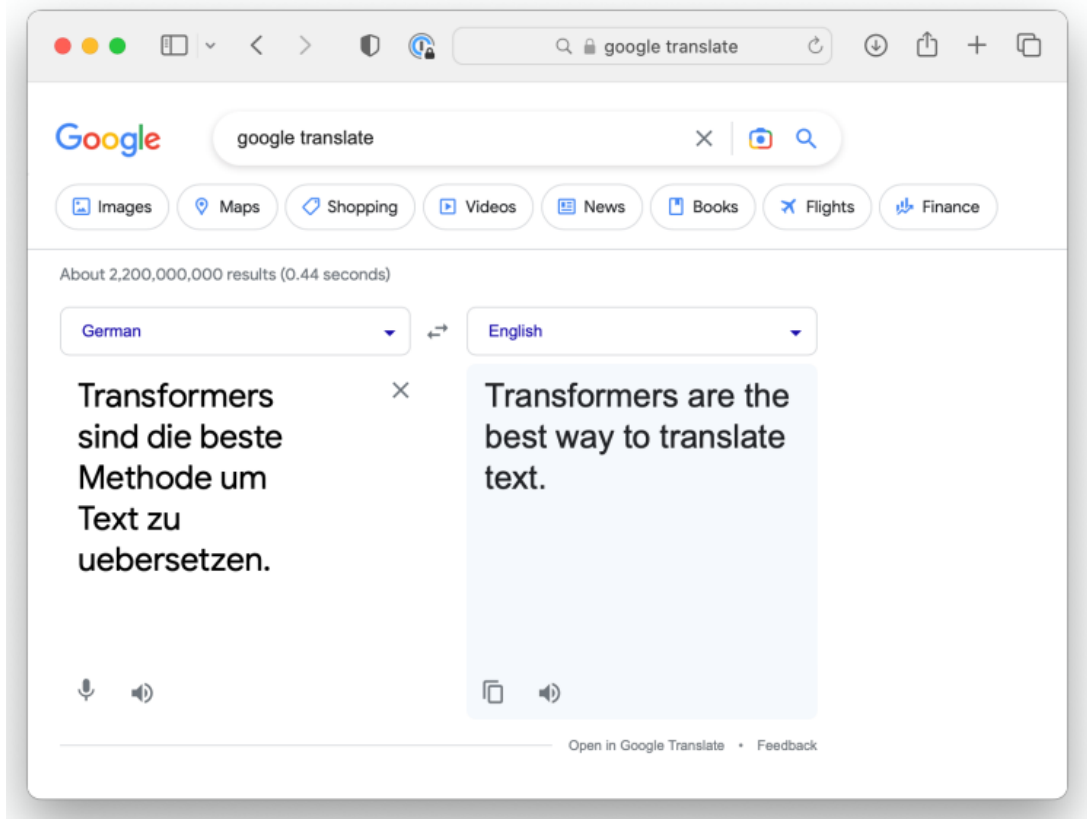
RNN



RNN rolled out over time



Introducción a Transformers



Introducción a Transformers

nature

View all journals Search Log in

Explore content ▾ About the journal ▾ Publish with us ▾

nature > articles > article

Download PDF

Article | Open Access | Published: 15 July 2021

Highly accurate protein structure prediction with AlphaFold

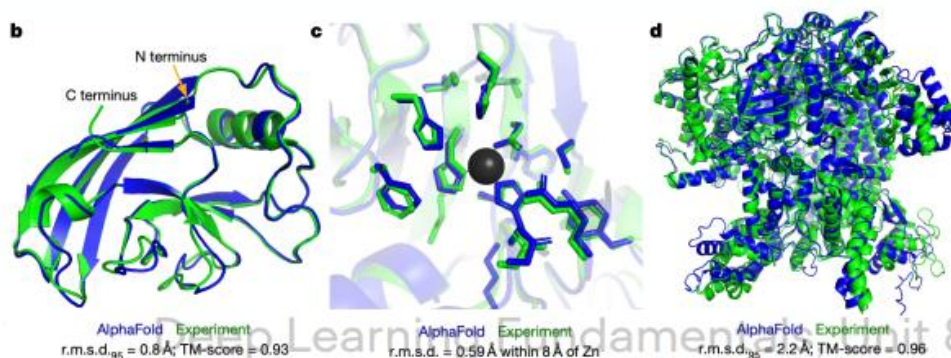
John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohli, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislaw Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, ... Demis Hassabis

Nature 596, 583–589 (2021) | Cite this article

923k Accesses | 4944 Citations | 3393 Altmetric | Metrics

Abstract

Proteins are essential to life, and understanding their structure can facilitate a mechanistic understanding of their function. Through an enormous experimental effort^{1,2,3,4}, the structures of around 100,000



Cornell University

We gratefully acknowledge support from the Simons Foundation and member institutions.

arXiv > cs > arXiv:1706.03762

Search... All fields Search

Help | Advanced Search

Computer Science > Computation and Language

[Submitted on 12 Jun 2017 (v1), last revised 6 Dec 2017 (this version, v5)]

Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from

Download:

- PDF
- Other formats (license)

Current browse context: cs.CL

< prev | next >

new | recent | 1706

Change to browse by: cs

cs.LG

References & Citations

- NASA ADS
- Google Scholar
- Semantic Scholar

82 blog links (what is this?)

DBLP – CS Bibliography

listing | bibtex

Ashish Vaswani
Noam Shazeer
Niki Parmar

Atención

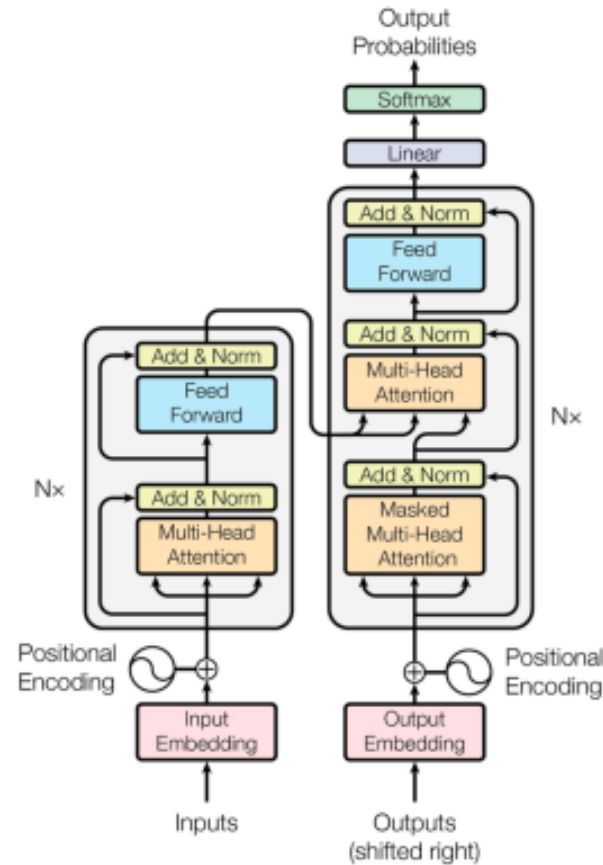
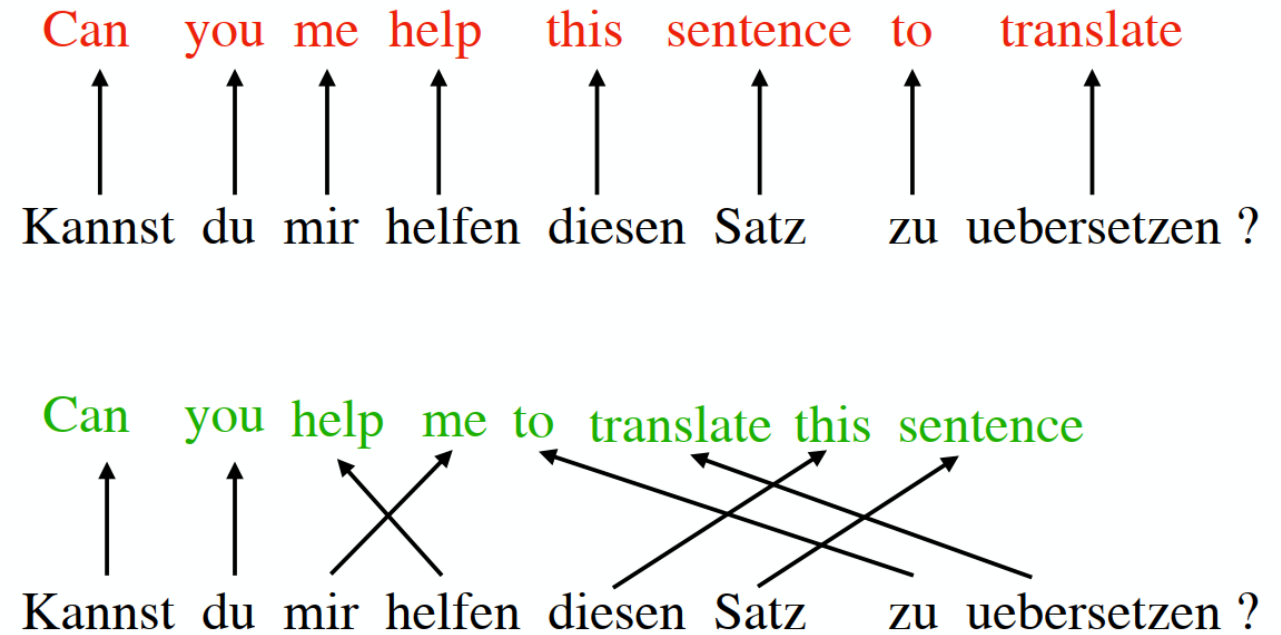


Figure 1: The Transformer - model architecture.

Nosotros no traducimos documentos palabra por palabra



Self-Attention

$$z^{(i)} = \sum_{j=1}^T \alpha_{ij} \cdot x^{(j)}$$

1. **Similarity** between i -th element all inputs $j = 1 \dots T$

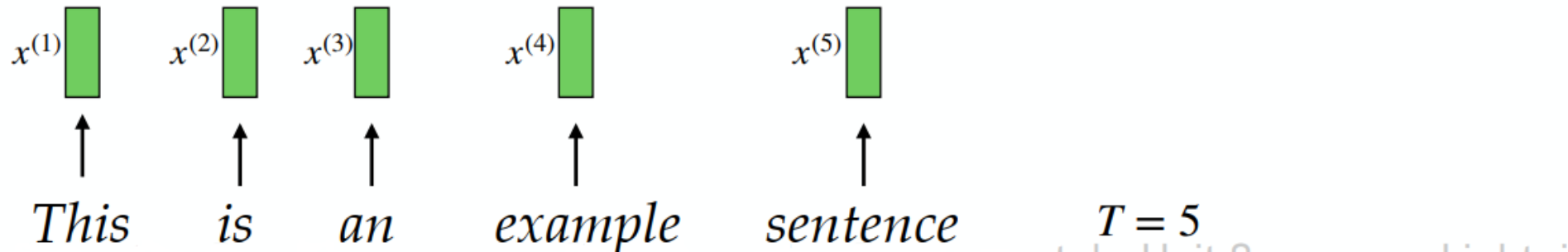
$$\omega_{ij} = x^{(i)\top} \cdot x^{(j)}$$

2. **Normalize** ω values to obtain attention scores α

So that attention value
sum to 1

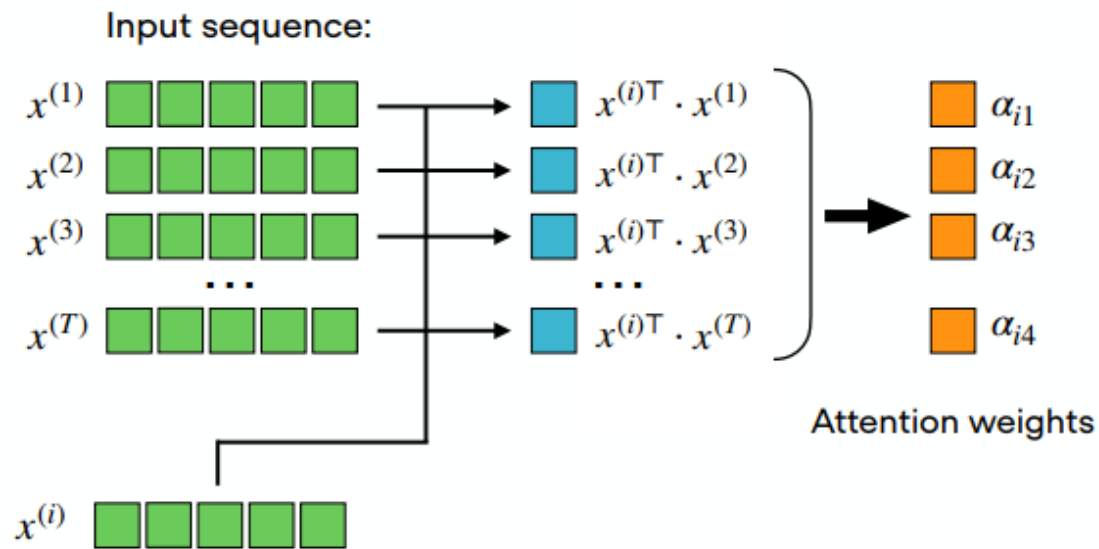
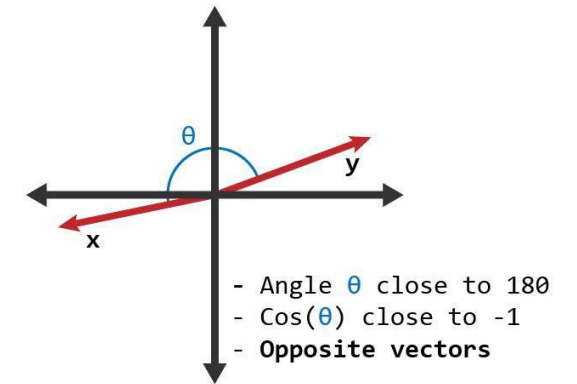
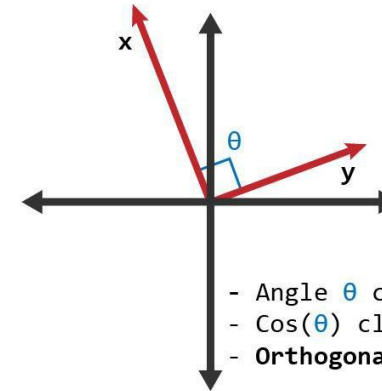
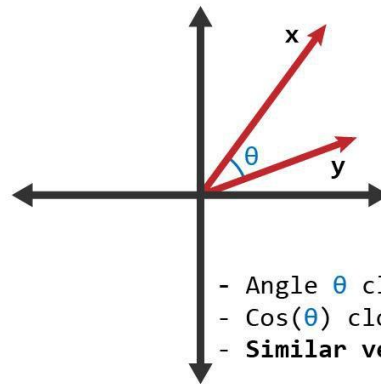
$$\sum_{j=1}^T \alpha_{ij} = 1$$

$$\alpha_{ij} = \frac{\exp(\omega_{ij})}{\sum_{j=1}^T \exp(\omega_{ij})} = \text{softmax} \left(\left[\omega_{ij} \right]_{j=1 \dots T} \right)$$



Producto punto: Similitud

$$\mathbf{x}^T \mathbf{y} = |\mathbf{x}| \cdot |\mathbf{y}| \cdot \cos(\theta)$$



$$z^{(i)} = \sum_{j=1}^T \alpha_{ij} \cdot x^{(j)}$$

$x^{(1)}$		$\times$		α_{i1}	
$+$	$x^{(2)}$		$\times$		α_{i2}
$+$	$x^{(3)}$		$\times$		α_{i3}
	$\dots$				
$+$	$x^{(T)}$		$\times$		α_{i4}

$=$

	$z^{(i)}$
--	-----------

Context vector

Self-attention with learnable weights

Idea: El modelo aprende qué tan importante es cada palabra. Modelo de atención más utilizado y propuesto en el paper original.

query sequence: $q^{(i)} = U_q x^{(i)}$ for $i \in [1, \dots, T]$

key sequence: $k^{(i)} = U_k x^{(i)}$ for $i \in [1, \dots, T]$

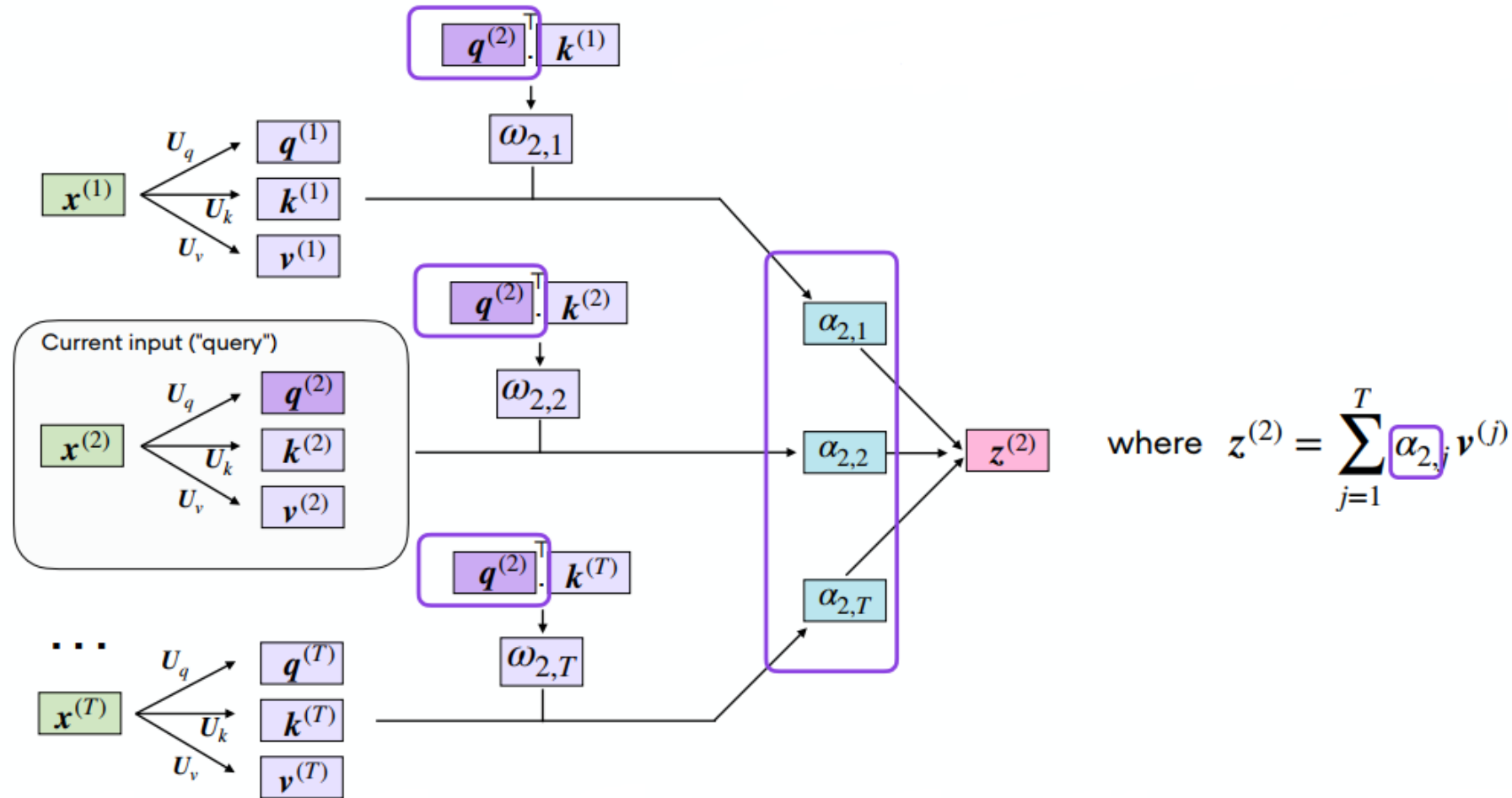
value sequence: $v^{(i)} = U_v x^{(i)}$ for $i \in [1, \dots, T]$

Query, key y value se inspiran en sistemas de recuperación de información.

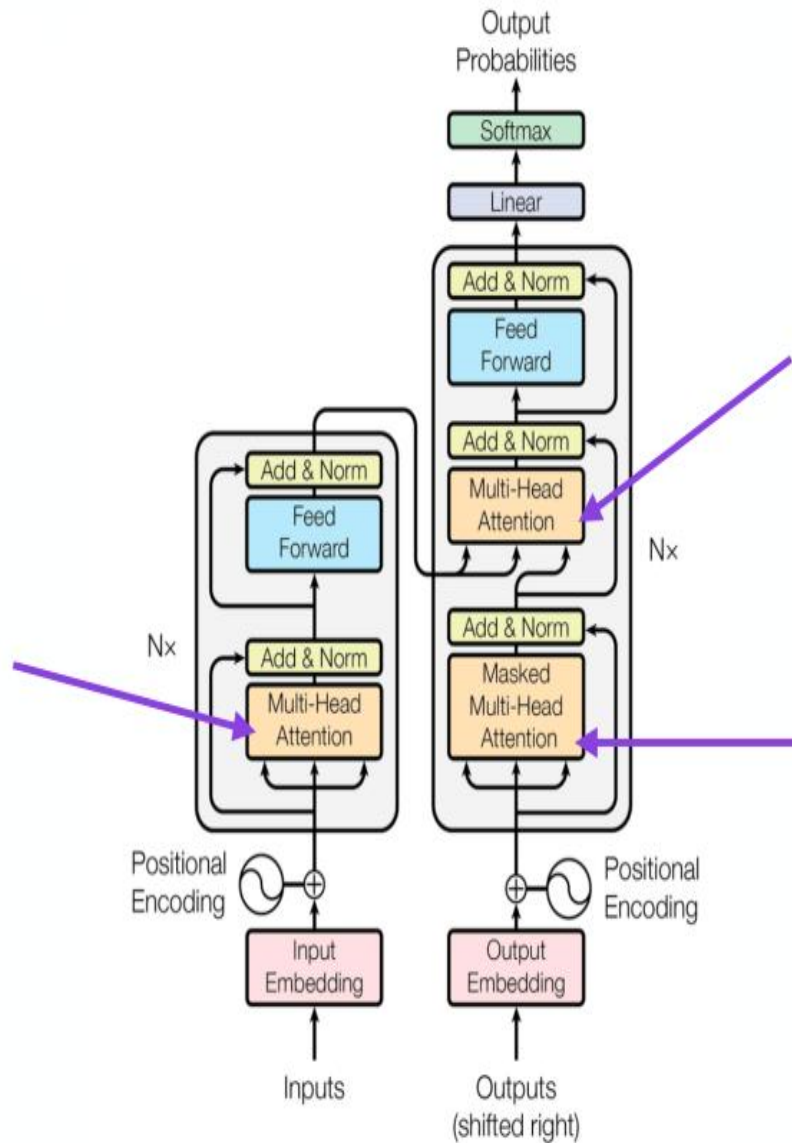
query se compara con **key** para devolver un **value**.

Al igual al módulo básico de self-attention, debemos calcular un **vector de contexto**

Self-attention with learnable weights



Self-attention with learnable weights



Resumen

Para cada token, el mecanismo de self-attention:

- Compara el query de cada palabra con los keys de todas las palabras en la secuencia de entrada
- Calcula el score de atención a partir de los valores obtenidos en la comparación
- Calcula el promedio ponderado de todas las entradas
- Metodología sequence-to-sequence
 - Toma T entradas
 - Devuelve T salidas

Multi-head attention

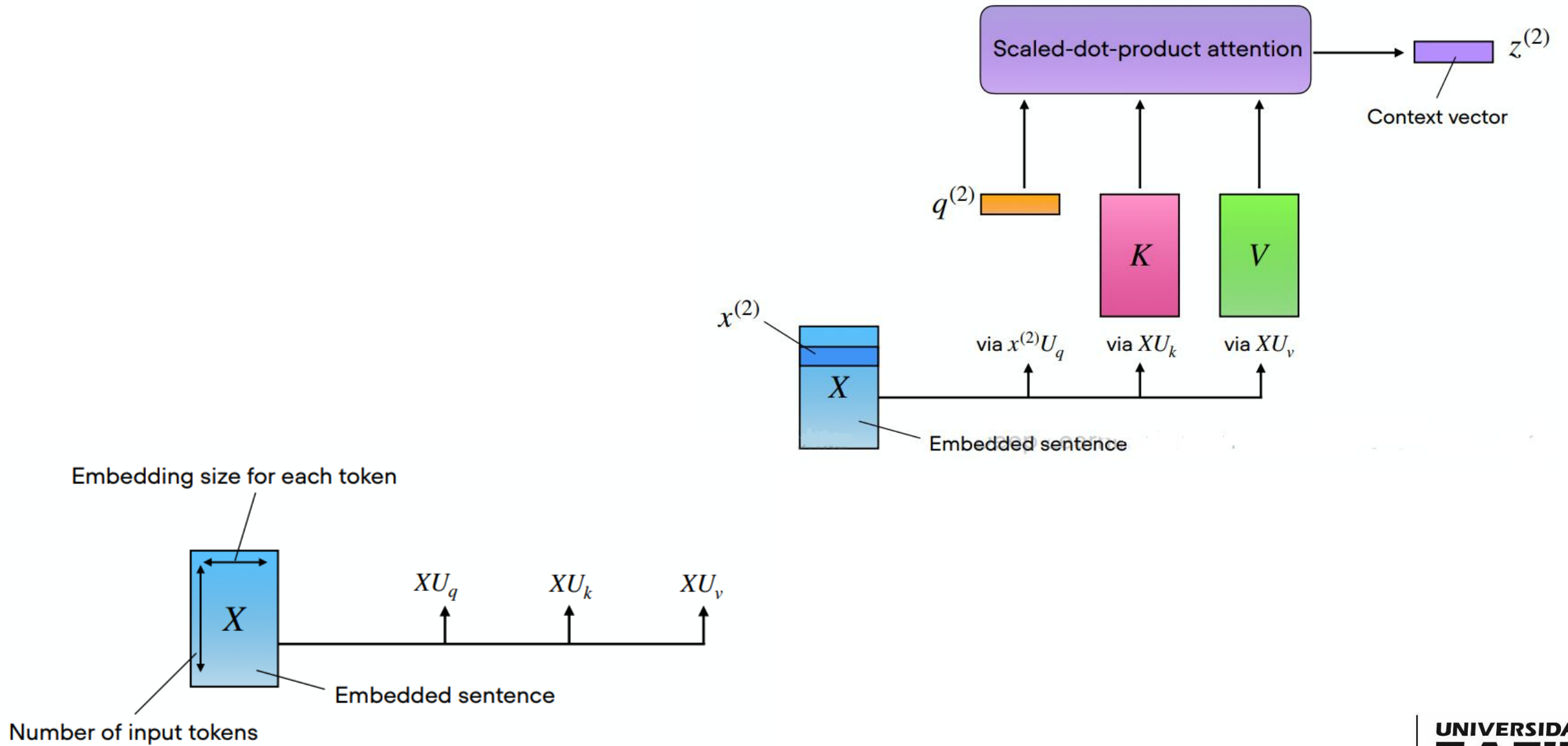
Anteriormente se usaron tres matrices de parámetros. Agreguemos un índice adicional

$$U_{q_1} \quad U_{k_1} \quad U_{v_1}$$

En multi-head attention tenemos un conjunto de matrices

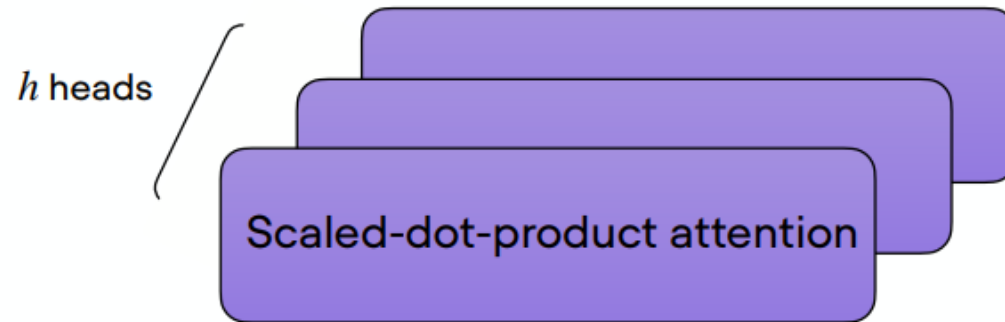
$$\begin{array}{ccc} U_{q_2} & U_{k_2} & U_{v_2} \\ U_{q_3} & U_{k_3} & U_{v_3} \\ \dots & & \end{array}$$

Multi-head attention

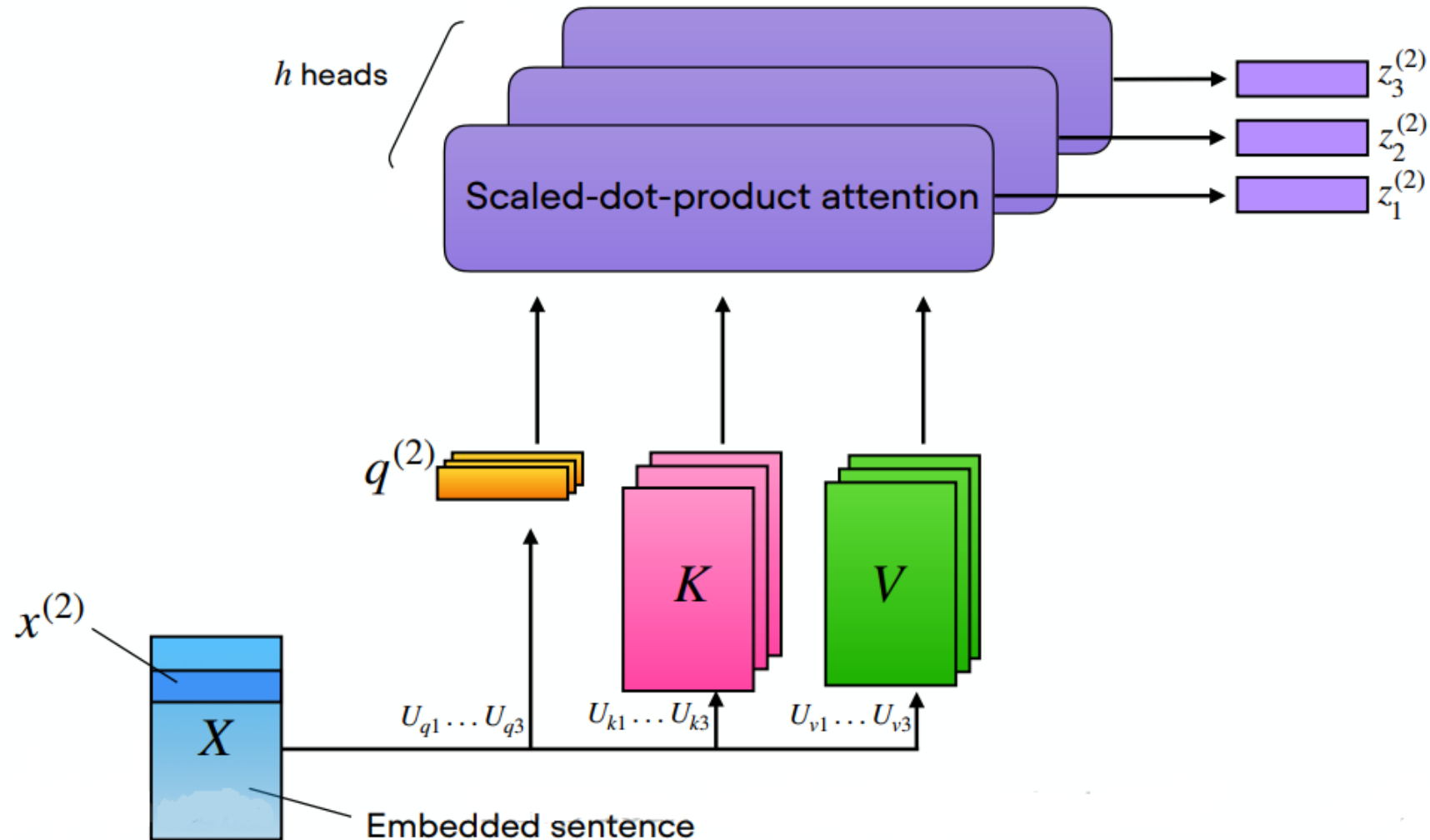


Multi-head attention

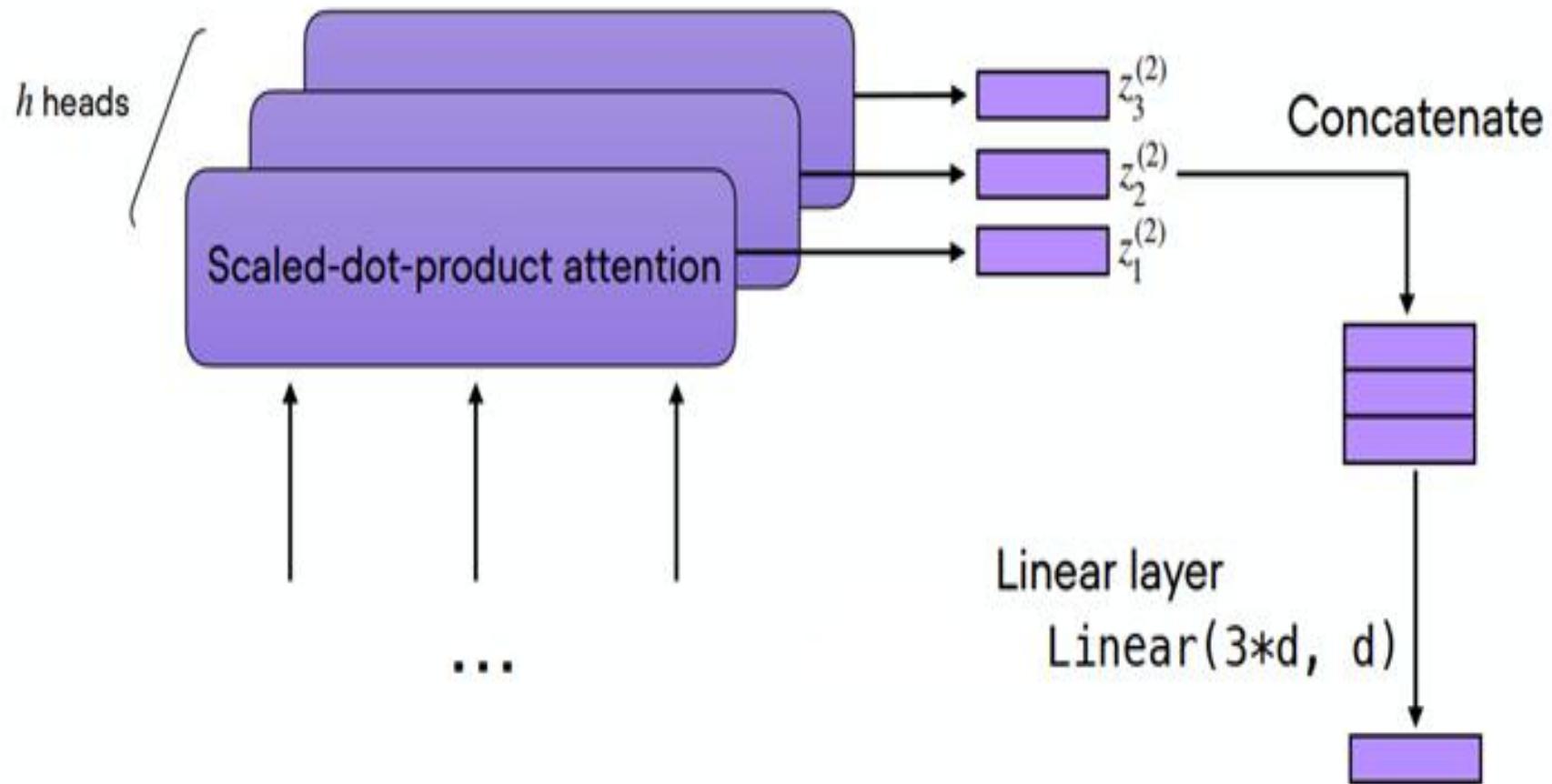
Similar a tener varios filtros convolucionales



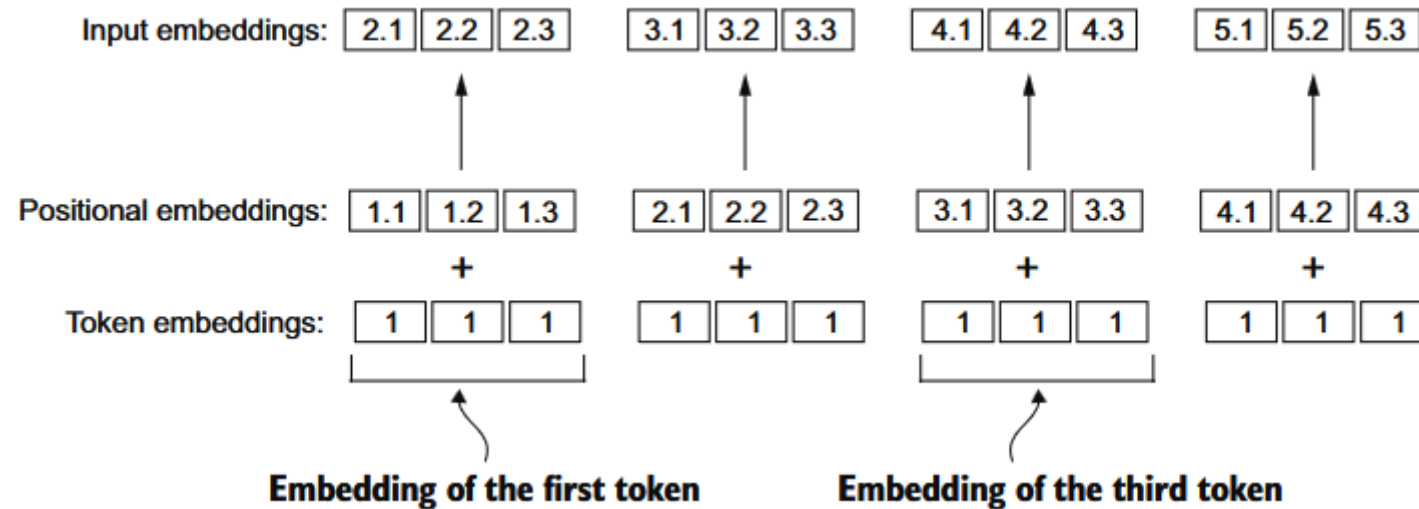
Multi-head attention



Multi-head attention



Encoding word positions



OpenAI: Positional embeddings are learnable parameters.

Llama, DeepSeek: Rotational Positional Embeddings – RoPE

Bert: Sinusoidal Positional Embeddings

Layer Normalization

- **Internal Covariate Shift** occurs when the activation distributions of neurons change across training steps, forcing frequent weight adjustments.
- This phenomenon slows down training by causing instability in optimization.
- It happens when earlier layers undergo significant updates, leading to drastic changes in the inputs of subsequent layers.

			f1	f2	f3	μ σ^2					f1	f2	f3
Item 1			a ₁	a ₂	a ₃	μ_1	σ_1^2	Item 1			a' ₁	a' ₂	a' ₃
Item 2								Item 2					
Item 3								Item 3					
Item 10								Item 10					

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \cdot \gamma + \beta$$

- **Batch Normalization** normalizes across **columns** (features).
- **Layer Normalization** normalizes across **rows** (data items).

Root Mean Square Layer Normalization

Biao Zhang¹ Rico Sennrich^{2,1}

¹School of Informatics, University of Edinburgh

²Institute of Computational Linguistics, University of Zurich

B.Zhang@ed.ac.uk, sennrich@cl.uzh.ch

4 RMSNorm

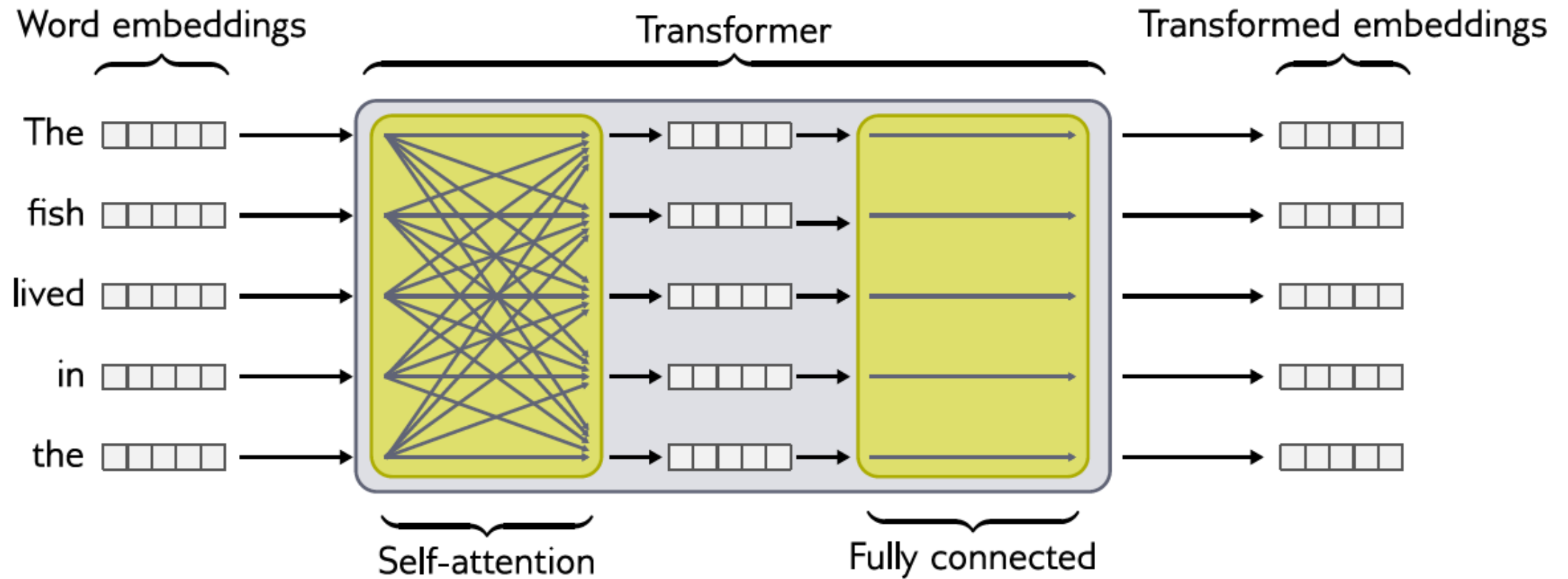
A well-known explanation of the success of LayerNorm is its re-centering and re-scaling invariance property. The former enables the model to be insensitive to shift noises on both inputs and weights, and the latter keeps the output representations intact when both inputs and weights are randomly scaled. In this paper, we hypothesize that the re-scaling invariance is the reason for success of LayerNorm, rather than re-centering invariance.

We propose RMSNorm which only focuses on re-scaling invariance and regularizes the summed inputs simply according to the root mean square (RMS) statistic:

$$\bar{a}_i = \frac{a_i}{\text{RMS}(\mathbf{a})} g_i, \quad \text{where } \text{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2}. \quad (4)$$

Intuitively, RMSNorm simplifies LayerNorm by totally removing the mean statistic in Eq. (3) at the cost of sacrificing the invariance that mean normalization affords. When the mean of summed inputs is zero, RMSNorm is exactly equal to LayerNorm. Although RMSNorm does not re-center

The transformer



Self supervised learning

1 - **Semi-supervised** training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.

Semi-supervised Learning Step

Model:



Dataset:



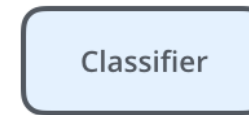
Objective:

Predict the masked word
(language modeling)

2 - **Supervised** training on a specific task with a labeled dataset.

Supervised Learning Step

Model:
(pre-trained
in step #1)

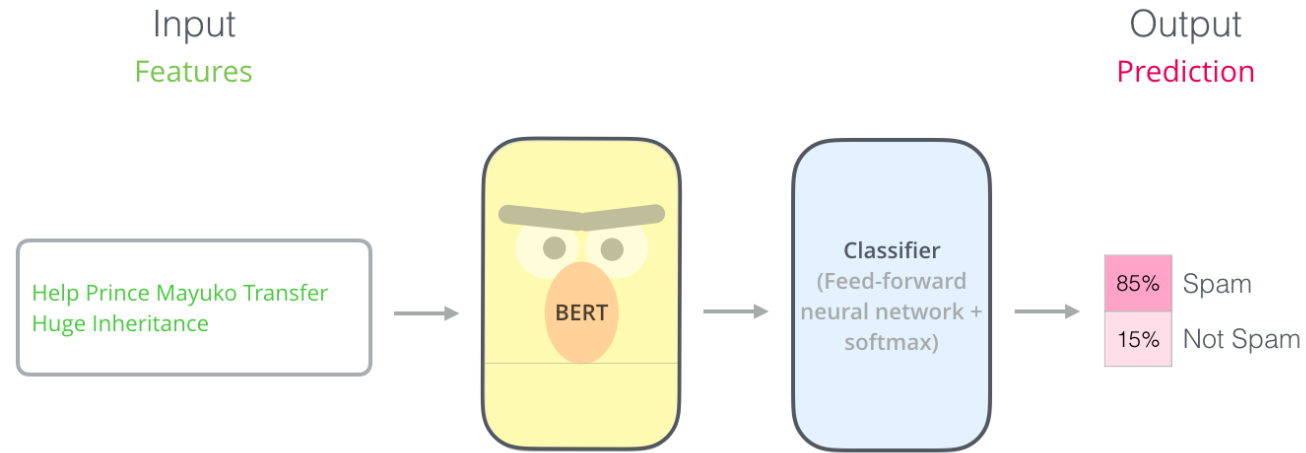


75% Spam
25% Not Spam

Dataset:

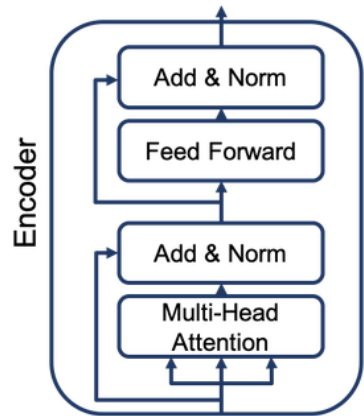
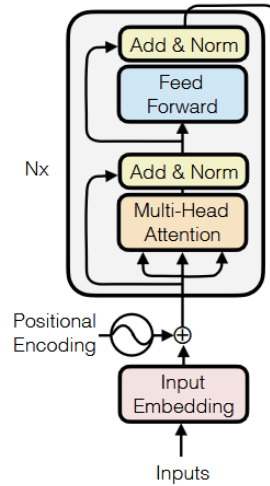
Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached...	Not Spam

Transfer learning

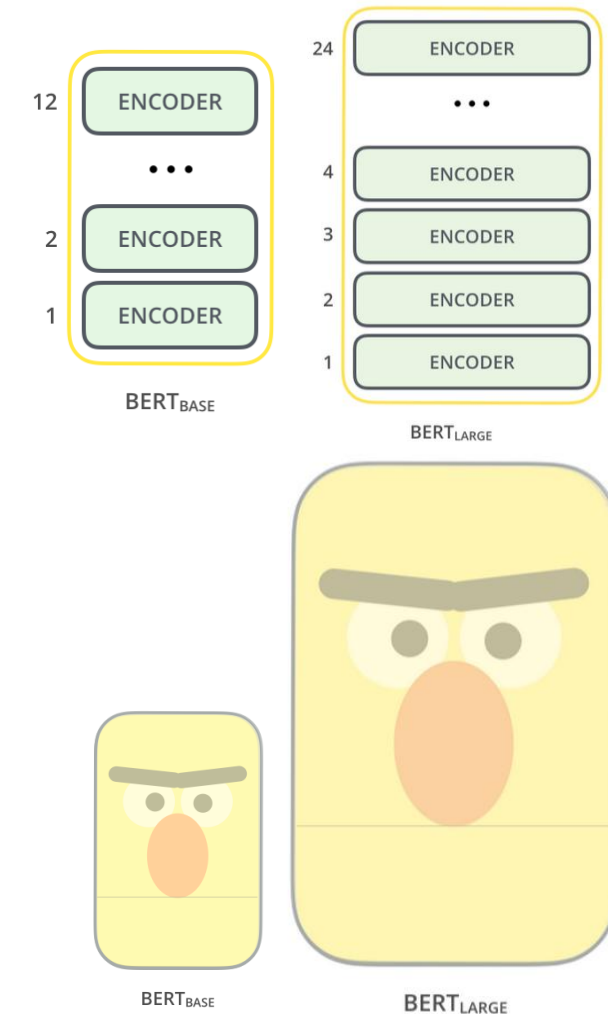


Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached...	Not Spam

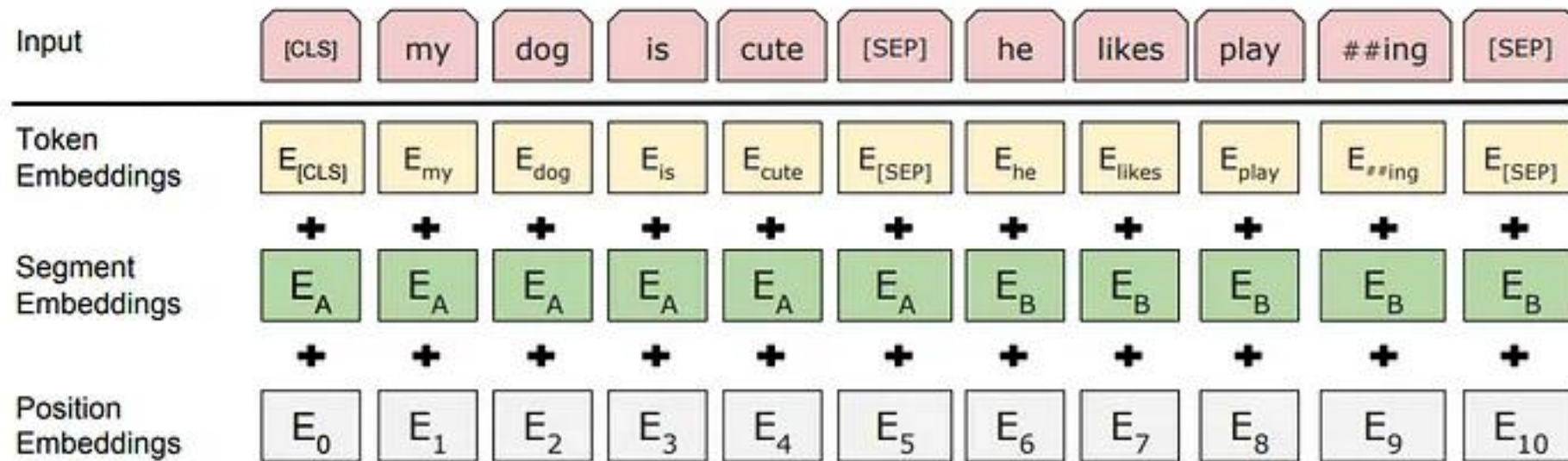
Model architecture



<code>bert-base-uncased</code>	12-layer, 768-hidden, 12-heads, 110M parameters. Trained on lower-cased English text.
<code>bert-large-uncased</code>	24-layer, 1024-hidden, 16-heads, 340M parameters. Trained on lower-cased English text.
<code>bert-base-cased</code>	12-layer, 768-hidden, 12-heads, 110M parameters. Trained on cased English text.
<code>bert-large-cased</code>	24-layer, 1024-hidden, 16-heads, 340M parameters. Trained on cased English text.
<code>bert-base-multilingual-uncased</code>	(Original, not recommended) 12-layer, 768-hidden, 12-heads, 110M parameters. Trained on lower-cased text in the top 102 languages with the largest Wikipedias (see details).
<code>bert-base-multilingual-cased</code>	(New, recommended) 12-layer, 768-hidden, 12-heads, 110M parameters. Trained on cased text in the top 104 languages with the largest Wikipedias (see details).



Model inputs



Pre-training: Masked Language Models

Use the output of the masked word's position to predict the masked word

Possible classes:
All English words

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzyva

FFNN + Softmax

15%

80%

my dog is [MASK]

10%

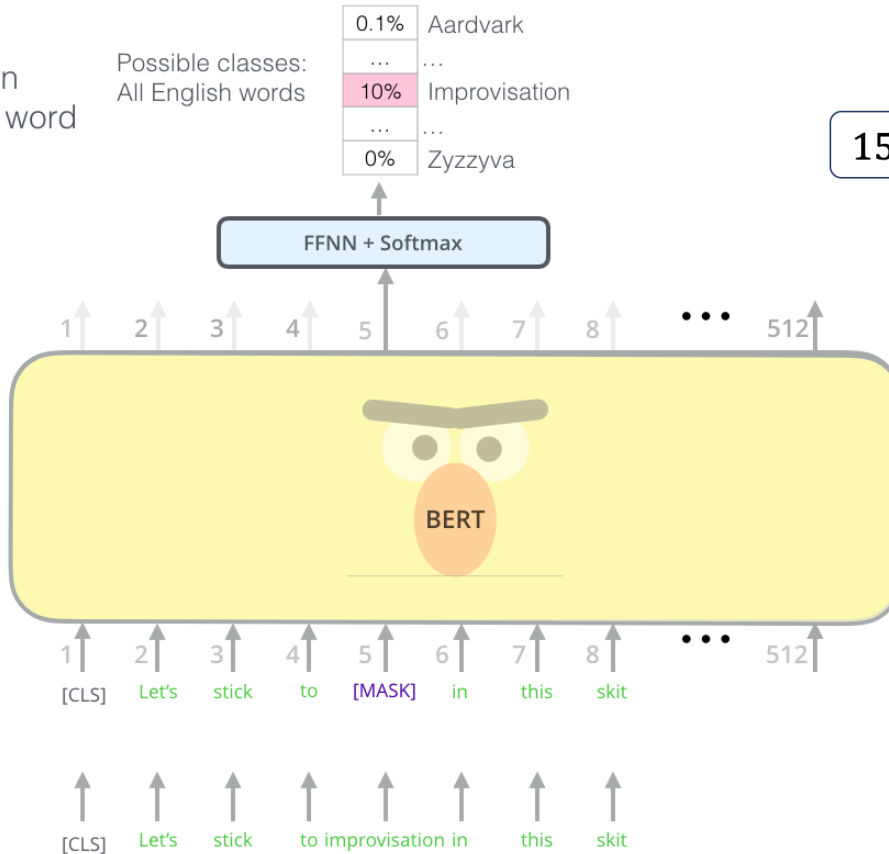
my dog is apple

10%

my dog is hairy

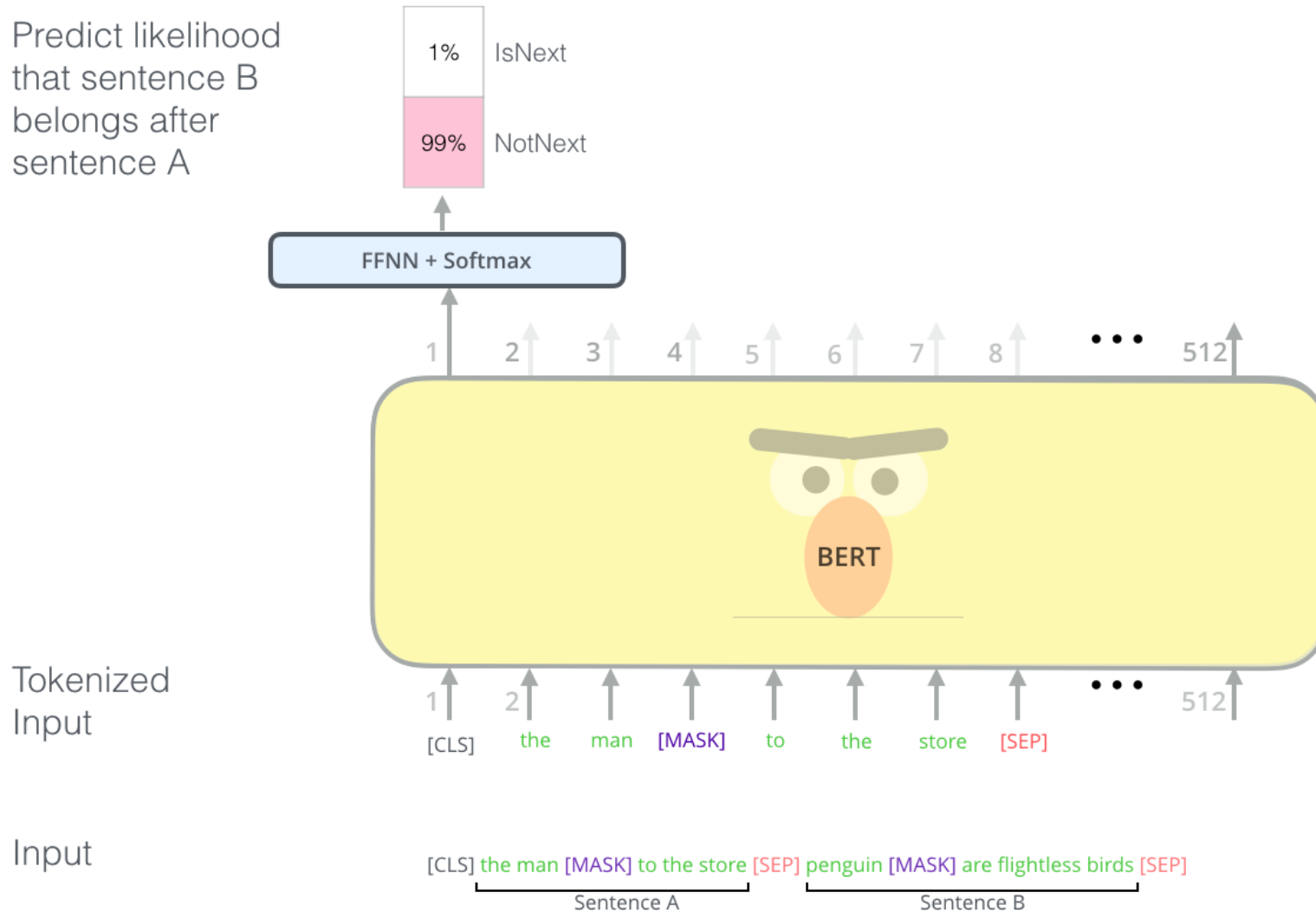
Randomly mask
15% of tokens

Input



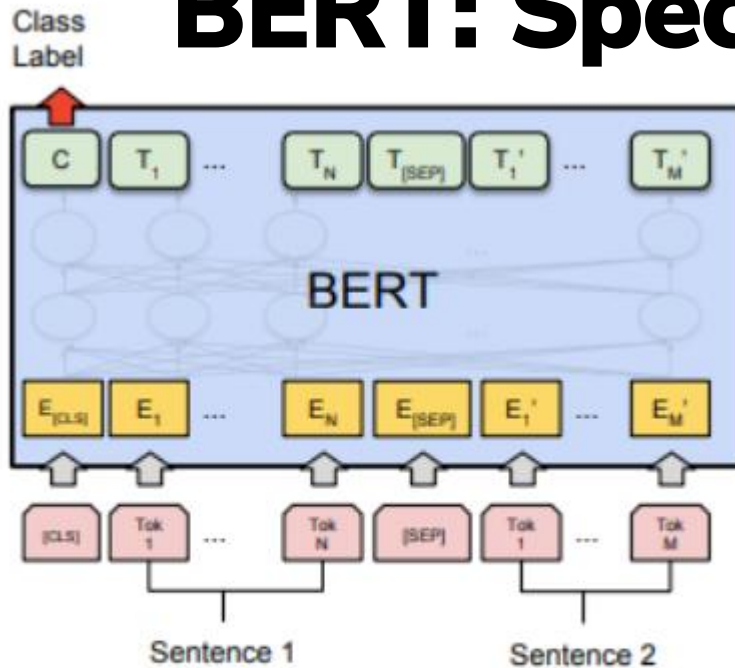
Pre-training: Two sentences tasks

Predict likelihood that sentence B belongs after sentence A



Input	[CLS] the man went to the [MASK] store [SEP] he bought a gallon [MASK] milk [SEP]
Label	IsNext
Input	[CLS] the man went to the [MASK] store [SEP] penguin [MASK] are flight ##less birds [SEP]
Label	NotNext

BERT: Specific Tasks



(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG

📌 **Objective:** Determine the relationship between two sentences.

✅ **Input:**

- Two sentences are concatenated with a [SEP] token in between.
- [CLS] is added at the beginning.
- Each token gets embeddings (**word, segment, positional**).

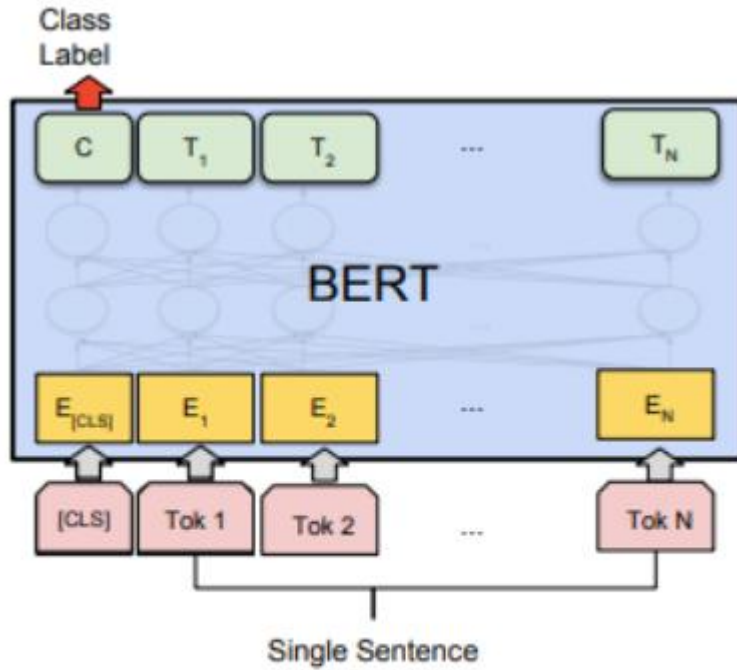
✅ **BERT Processing:**

- Applies **self-attention** over both sentences.
- [CLS] token is used to represent the **entire input**.

✅ **Output:**

- A **classifier** (fully connected layer + softmax) is applied to the [CLS] token to predict the **relationship** between sentences (e.g., entailment, contradiction).

BERT: Specific Tasks



(b) Single Sentence Classification Tasks:
SST-2, CoLA

 **Objective:** Classify the **entire sentence** into a category (e.g., sentiment analysis).

 **Input:**

- A single sentence with a [CLS] token at the start.

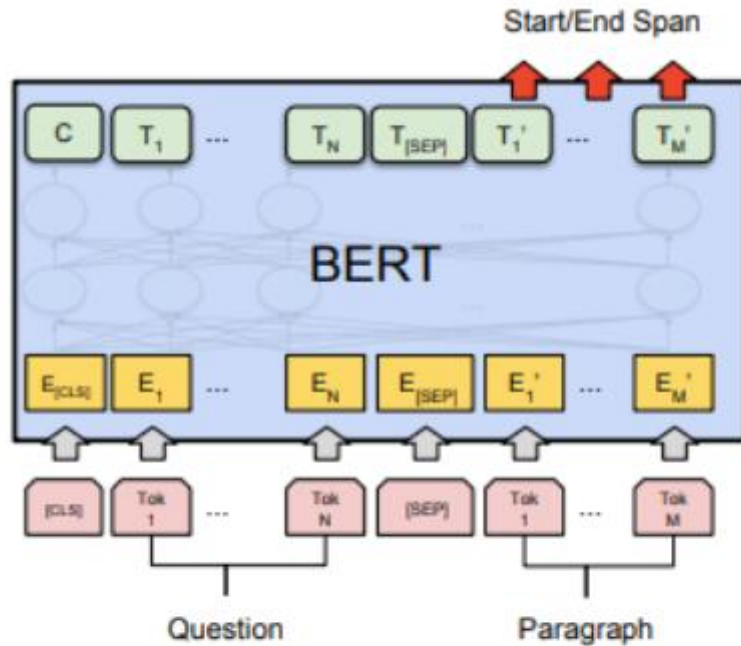
 **BERT Processing:**

- Self-attention processes the sentence.

 **Output:**

- The classifier is applied to the [CLS] token, which encodes sentence-level information.

BERT: Specific Tasks



(c) Question Answering Tasks:
SQuAD v1.1

📌 **Objective:** Find the **start** and **end** of the answer span in a paragraph given a question.

✅ **Input:**

- The **question** and **paragraph** are separated by [SEP].

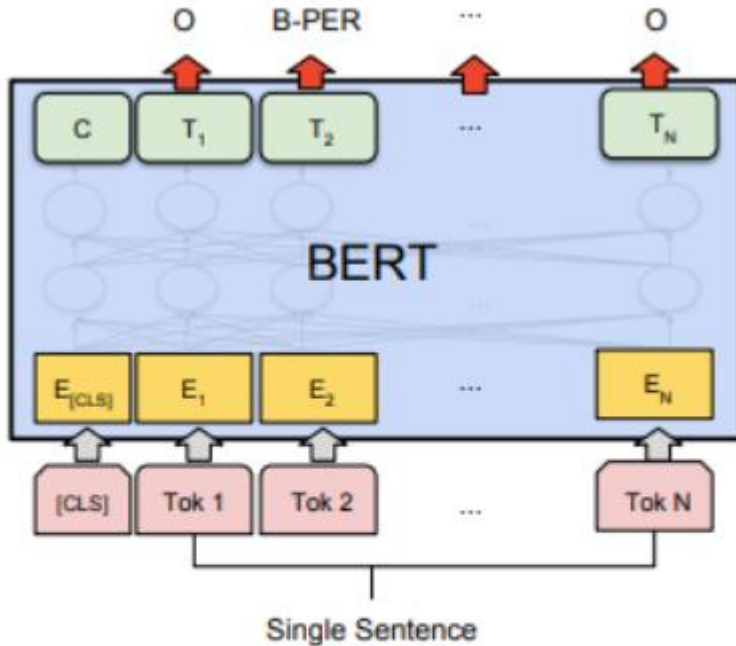
✅ **BERT Processing:**

- Self-attention captures relationships between the **question** and the **paragraph**.

✅ **Output:**

- Two classifiers predict:
 1. **Start position** of the answer.
 2. **End position** of the answer.

BERT: Specific Tasks



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

📌 **Objective:** Assign a **label** to each token in the sentence (e.g., Person, Organization, Location).

✅ **Input:**

- A single sentence with a [CLS] token

✅ **BERT Processing:**

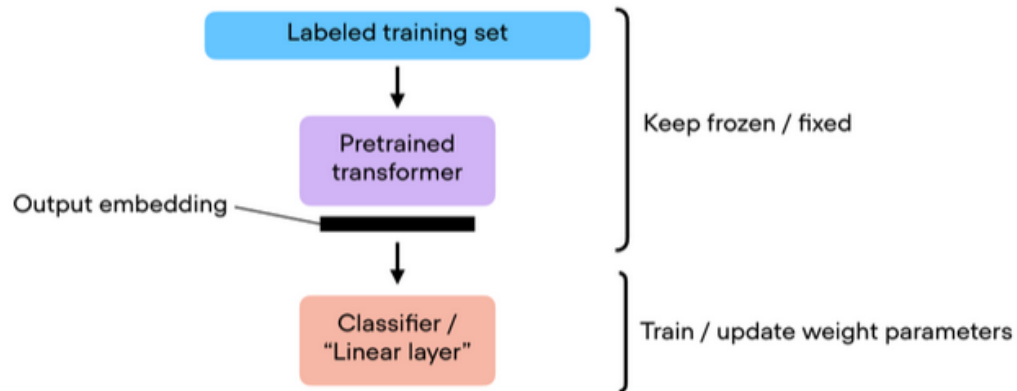
- Self-attention captures context for each Word

✅ **Output:**

- Instead of using the [CLS] token, **each token** has its **own classification**.
- Labels are assigned to **each token** (e.g., "B-PER" for "beginning of a person's name").

Utilización de Transformers pre-entrenados

1. Feature-based approach



2. Fine-tuning approach

